But, just as we think we've wrapped up this functionality, our customer wanders over with a copy of *Information Technology and Golf Weekly*. Apparently, it has an article about the HotWired style of browser interface, where stuff gets updated on the fly. Hmmm…let's look at that tomorrow.

Playtime

Here's some stuff to try on your own:

- Create a migration that copies the product price into the line item, and change the `add_product` method in the `Cart` model to capture the price whenever a new line item is created. Add prices to the `line_items.yml` fixture.

- Write unit tests that add both unique products and duplicate products to a cart. Assert how many products should be in the cart in each instance. Note that you'll need to modify the fixture to refer to products and carts by name —for example, `product: ruby`.

- Check products and line items for other places where a user-friendly error message would be in order.

- Add the ability to delete individual line items from the cart. This will require buttons on each line, and such buttons will need to be linked to the `destroy` action in the `LineItemsController`.

- We prevented accessing other users' carts in the `LineItemsController`, but you can still see other carts by navigating directly to a URL like `http://localhost/carts/3`. See if you can prevent accessing any cart other than the one currently stored in the session.

Footnotes

[48] http://www.fileformat.info/info/unicode/char/00d7/index.htm

[49] http://guides.rubyonrails.org/debugging_rails_applications.html#the-logger

[50] http://gnuwin32.sourceforge.net/packages/coreutils.htm